



**Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Monterrey**

Escuela de Ingeniería y Ciencias

Ingeniería en Ciencias de Datos y Matemáticas

Uso de álgebras modernas para seguridad y criptografía, MA2006B.604

Reporte Técnico - Etapa 1: Gestor de Identidades

Equipo 2:

Ana Lidia Hernández Díaz	A00838643
Ana Paula García Valverde	A01174572
María Fernanda Montoya López	A01743214
Javier Rojas Orrante	A01352213
Xavier Lago Hicks	A01403177

Profesores:

Raúl Gómez Anas Wajid Alberto F. Martínez

*Proyecto: Firmas Solidarias – Tecnología Criptográfica para los Derechos Humanos
Socio Formador: Casa Monarca*

Monterrey, Nuevo León. 30 de abril de 2026

Índice

1. Introducción	3
1.1. Descripción de la Problemática	3
2. Marco de referencia	3
2.1. Fundamentos de Criptografía de Clave Pública	3
2.1.1. Algoritmo RSA	3
2.1.2. Pilares de la Seguridad	4
2.2. Herramientas y Bibliotecas de Implementación	4
2.2.1. FastAPI como orquestador del Backend	4
2.2.2. JWT (JSON Web Tokens) como guardián de Sesiones	4
2.2.3. itsdangerous como el guardián de Sesiones	5
2.2.4. Encriptación	5
2.3. Referentes Operativos y Soluciones Existentes	5
2.4. Marco Legal y Ético	5
2.4.1. Derechos ARCO	5
2.4.2. Ley Federal de Protección de Datos Personales	6
2.4.3. Ética de la Ciberseguridad	6
3. Solución Propuesta y Arquitectura	6
3.1. Estructura de las Capas de Seguridad	6
3.1.1. Capa 1: Autenticación Bifurcada y Puerta de Entrada	7
3.1.2. Capa 2: Control de Acceso Basado en Roles (RBAC)	7
3.1.3. Capa 3: Validación Estructural de Datos y Contratos de Entrada	8
3.1.4. Capa 4: Persistencia y Bóveda Criptográfica (Datos en Reposo)	8
3.2. Flujo de una Petición a Través de las Capas (Caso de Éxito vs Error)	9
3.3. Justificación	9
3.3.1. Viabilidad de Infraestructura: Despliegue en Alojamiento Compartido (HostGator) vs. Nube Compleja (AWS)	9
3.3.2. Robustecimiento del Esquema de Autenticación mediante Bifurcación de Acceso (RBAC)	10
3.3.3. Adopción de Estándares Algorítmicos Criptográficos e Industriales	10
3.3.4. Filosofía del Ciclo de Vida: El Principio de Entrega Única (Single Delivery)	10
3.4. Implementación Técnica y Estructura del Software	11
3.4.1. BackEnd	11
3.4.2. FrontEnd	12
3.5. Conexión de los Componentes	13
4. Resultados	13
4.1. Estructura de la solución	13
4.2. Resultados del BackEnd y API	14
4.3. Resultados del FrontEnd e Interfaz	14
4.4. Resultados de la Solución Completa	14
4.5. Validación mediante Casos de Prueba	15
5. Conclusiones	15
5.1. Trabajo Futuro	16
6. Anexos	18
6.1. Tabla de casos de prueba	18

Índice de figuras

1.	Arquitectura del sistema (elaboración propia)	6
2.	Panel de Control del administrador (elaboración propia)	12
3.	Solicitud de credenciales (elaboración propia)	13
4.	Portal del Usuario (elaboración propia)	13

Índice de tablas

1.	Resumen de validaciones (elaboración propia)	15
2.	Evidencia de casos de pruebas del sistema (elaboración propia)	18
2.	Evidencia de casos de pruebas del sistema (elaboración propia)	19

Abstract

Este reporte documenta el diseño e implementación de la primera etapa del reto “Gestor de Identidades” desarrollado para la organización Casa Monarca. El objetivo central fue establecer un sistema de manejo centralizado y seguro para las identidades digitales de los colaboradores, asegurando la protección de datos sensibles de la población migrante atendida. La solución técnica emplea una arquitectura de Control de Acceso Basado en Roles (RBAC) construida con el lenguaje Python y el framework FastAPI, utilizando una base de datos MySQL para la persistencia. El sistema gestiona el ciclo de vida completo de las identidades, incluyendo los procesos de alta, activación, revocación y expiración automática; y garantiza el no repudio mediante la generación de bitácoras de auditoría inmutables. Finalmente, el desarrollo se alineó con el marco legal mexicano de los Derechos ARCO y la Ley de Firma Electrónica Avanzada, priorizando un equilibrio ético entre la robustez criptográfica y la usabilidad para el personal operativo de la organización.

1. Introducción

El presente reporte brinda una solución a la problemática de gestión de identidades, la cual se desarrolla para la organización socioformadora Casa Monarca. El objetivo principal es brindar protección de la información sensible de las personas migrantes que reciben apoyo de la organización, esto por medio de un manejo centralizado y seguro de las identidades de los colaboradores a través de certificados digitales.

El diseño de la solución busca alinear con el marco legal mexicano, cumpliendo con los Derechos ARCO y la Ley Federal de Protección de Datos Personales en Posesión de Particulares. El enfoque técnico busca priorizar la usabilidad y la eficiencia operativa, esto con el objetivo de mejorar la productividad sin exigir conocimientos técnicos avanzados al personal de la organización. Por otro lado, se asegura la integridad, autenticidad y el no repudio en la gestión de documentos y accesos a través de una arquitectura de Control de Acceso Basado en Roles (RBAC).

1.1. Descripción de la Problemática

Dado a que Casa Monarca maneja información sensible de personas migrantes en situación vulnerable, las principales anomalías que el sistema busca impedir son la suplantación de identidad, la alteración de datos y la falta de trazabilidad en el acceso a la información. De acuerdo a protocolos de seguridad modernos, se requiere un ciclo de vida completo para los usuarios donde exista un alta, baja, revocación y rastreo de estos [1].

2. Marco de referencia

2.1. Fundamentos de Criptografía de Clave Pública

La base de la solución reside en la criptografía asimétrica, la cual utiliza pares de claves, siendo la pública y la privada, para garantizar la seguridad en entornos de red. Es importante mencionar el uso de algoritmos criptográficos como el RSA.

2.1.1. Algoritmo RSA

Es un algoritmo que permite el intercambio seguro de información y la generación de firmas digitales, por lo cual es lo que garantiza el no repudio y la autenticidad. Además, es considerado el estándar para asegurar que solo el poseedor de la clave privada pueda descifrar información o generar firmas digitales auténticas. Por medio de la generación de llaves, donde se crean dos números primos grandes y mediante operaciones matemáticas, se derivan la llave pública (n, e) y la privada (n, d) . Para cifrarlo, el emisor toma el mensaje (convertido a número) y lo eleva a la potencia e módulo n y para descifrarlo, el receptor toma el criptograma y lo eleva a la potencia d módulo n para recuperar el mensaje original.

2.1.2. Pilares de la Seguridad

La implementación busca cumplir con los tres pilares de la seguridad: la confidencialidad, integridad y disponibilidad. La confidencialidad responde a la problemática del acceso no autorizado, en este contexto, que la información esté disponible únicamente a colaboradores de Casa Monarca. Esto resulta relevante debido a la información sensible que se maneja dentro de la organización y si llegara a las manos equivocadas se vulneraría los derechos de las personas migrantes.

Por otro lado la integridad, definida como el “estado de lo que está completo o tiene todas sus partes” [2], garantiza que la información sea correcta y no modificada, lo cual implica que la información esté tal cual como fue creada y no exista algún tipo de alteración por parte de terceros. Una cualidad muy importante para este pilar es la autenticación, la cual identifica a la persona que creó la información.

Por último, la disponibilidad garantiza que se pueda acceder a la información cuando se requiera. En este sentido, se busca que exista un balance entre este pilar y la confidencialidad, ya que a menudo suelen conflictuarse debido a que un cifrado complejo puede dejar de ser accesible.

2.2. Herramientas y Bibliotecas de Implementación

Para la construcción del sistema en Python, se seleccionaron herramientas que permiten aplicar estos conceptos matemáticos de forma eficiente.

La biblioteca cryptography como el motor criptográfico

Esta librería es el estándar de la industria en Python utilizado para realizar operaciones criptográficas y de firma digital. En el proyecto, su uso se centraliza en dos frentes críticos:

- **Infraestructura de Clave Pública (PKI):** Se utiliza para modelar la Autoridad Certificadora (CA) interna, permitiendo generar pares de llaves asimétricas RSA de 2048 bits con un exponente público de 65537. Asimismo, gestiona la construcción, codificación y firma de certificados digitales estándar X.509 para administradores y coordinadores.
- **Protección de Material Criptográfico:** Emplea el algoritmo de empaquetado seguro PKCS8 con el método `BestAvailableEncryption` (el cual aplica AES-256 según el OpenSSL local) para cifrar la llave privada del usuario con su contraseña antes de estar disponible para su descarga.

El hashing y verificación de contraseñas no utiliza librerías externas de terceros, sino que se ejecuta a través del módulo nativo `hashlib.pbkdf2_hmac` de Python, configurado de manera robusta bajo PBKDF2-HMAC-SHA256 con 120,000 iteraciones y sales de 16 bytes generadas por `secrets.token_bytes`.

2.2.1. FastAPI como orquestador del Backend

FastAPI se encarga de unir la seguridad con la lógica de negocio definiendo las rutas para el ciclo de vida del usuario: Alta, Baja, Revocación y Rastreo, por medio de la gestión de endpoints. Además, se utilizará inyección de dependencias para validar automáticamente el actor que realiza una acción antes de procesar cualquier lógica, asegurando que solo usuarios autenticados lleguen a la base de datos.

- **Validación de Datos:** Gracias a su integración nativa, asegura que toda información que entra al sistema cumpla con el formato correcto antes de ser procesada por los servicios criptográficos.

2.2.2. JWT (JSON Web Tokens) como guardián de Sesiones

El manejo de identidades digitales requiere una forma segura de transmitir quién es quién sin exponer credenciales constantemente. Es por esto que tras una autenticación exitosa (MFA), el sistema genera un JWT firmado, este token viaja en cada petición del usuario y contiene sus privilegios y rol. Con el middleware de Autorización, el sistema revisa el JWT en cada paso para determinar si el rol asociado tiene el permiso necesario (ej. `users:revoke`) para ejecutar la acción solicitada.

2.2.3. itsdangerous como el guardián de Sesiones

El manejo de identidades digitales en la capa web no utiliza JWT (JSON Web Tokens), sino cookies de sesión firmadas criptográficamente para evitar la exposición constante de credenciales. Tras una autenticación exitosa, el servidor utiliza la clase `URLSafeTimedSerializer` provista por la biblioteca `itsdangerous` para generar un token compacto firmado con la clave secreta global (`session_secret`) y el salt del sistema. Este token se inyecta en el navegador a través de la cookie segura `cm_session`. En cada petición HTTP entrante, el servidor intercepta la cookie de forma obligatoria, verifica la integridad de su firma HMAC en tiempo constante y valida que no haya superado su tiempo de expiración máximo configurado de 8 horas.

2.2.4. Encriptación

Para proteger la información almacenada en la base de datos, el proyecto utiliza la librería `cryptography`, específicamente `Fernet`, como mecanismo principal de cifrado simétrico. Su función es convertir los datos sensibles en texto cifrado antes de guardarlos y descifrarlos automáticamente cuando el sistema necesita leerlos. Esta lógica se implementa mediante las clases `EncryptedText` y `EncryptedJSON`, que funcionan como tipos personalizados de SQLAlchemy. Su propósito es que campos como nombres, correos, certificados, llaves privadas, contraseñas hashadas, registros de auditoría y metadatos no queden almacenados en texto plano dentro de la base de datos.

Además, el sistema usa herramientas como `hashlib`, `hmac` y `base64` para reforzar la protección y compatibilidad de los datos. `hashlib` permite derivar material criptográfico a partir de una clave secreta del sistema, mientras que `hmac` genera identificadores de búsqueda seguros para poder consultar ciertos campos cifrados sin revelar su valor original. `base64` se usa para transformar datos binarios en texto compatible con almacenamiento en base de datos. En conjunto, estas herramientas permiten que la base de datos conserve funcionalidad operativa, como búsquedas e índices, pero reduciendo el riesgo de exposición si alguien accede directamente a las tablas.

2.3. Referentes Operativos y Soluciones Existentes

Tras una exhaustiva investigación acerca de los referentes operativos y las soluciones existentes, se analizaron modelos de gestión de identidades para adaptar sus mejores prácticas al contexto de la ONG. El principal referente es el sistema del SAT, del cual se tomó como referencia su modelo de administración de contribuyentes en México, el cual utiliza certificados digitales para vincular una identidad legal con un usuario digital, asegurando la trazabilidad de las acciones. Asimismo, se exploró el sistema de gestión de roles de la plataforma Canvas, la cual administra la vista del portal dependiendo del nivel del usuario. Como soluciones existentes, se investigó la gestión del Ciclo de Vida, que de acuerdo con IBM se encarga de monitorear los datos a nivel de archivo, gestionando los archivos en función del tipo, el tamaño y la antigüedad [3]. De este modo, siguiendo los estándares de la literatura, el sistema implementa las etapas de Alta, Baja, Revocación y Rastreo de identidades digitales.

2.4. Marco Legal y Ético

La solución técnica responde a un entorno normativo y humano riguroso, asegurando que la protección de datos sensibles se realice bajo estándares éticos y legales mexicanos.

2.4.1. Derechos ARCO

El diseño del sistema garantiza el cumplimiento de los derechos de Acceso, Rectificación, Cancelación y Oposición del usuario sobre su identidad digital.

- **Acceso:** Se facilita a través del Portal de Autoservicio, donde cada colaborador puede visualizar su información personal, el estado de su cuenta y la vigencia de sus certificados activos.
- **Rectificación:** El Administrador dispone de herramientas integradas en el Dashboard para corregir nombres, correos electrónicos o modificar roles según cambien las responsabilidades del personal.
- **Cancelación y Oposición:** Estos derechos se materializan mediante el sistema de revocación manual y la expiración automática de cuentas. Al marcar a un usuario como `revoked` o `expired`, el sistema bloquea de forma inmediata e irreversible el acceso, garantizando que los datos no sean tratados más allá del tiempo autorizado por el titular.

2.4.2. Ley Federal de Protección de Datos Personales

La arquitectura de seguridad por capas cumple con las medidas técnicas exigidas por la legislación mexicana para proteger la información de poblaciones vulnerables. El uso de hashing robusto mediante **PBKDF2-HMAC-SHA256** con 120,000 iteraciones asegura que las contraseñas nunca se almacenen en texto plano. Además, el material criptográfico de mayor sensibilidad, como las llaves privadas, se somete a un cifrado simétrico **AES-256** antes de su distribución. Para cumplir con el principio de responsabilidad y trazabilidad, el `AuditService` registra de manera inmutable cada interacción crítica, documentando el actor, la acción, la dirección IP y el resultado, permitiendo auditorías forenses.

2.4.3. Ética de la Ciberseguridad

Se prioriza un equilibrio entre la robustez técnica y la usabilidad para el personal humanitario de Casa Monarca, evitando barreras tecnológicas que entorpezcan su misión. Esto se logra mediante una autenticación bifurcada: mientras que los roles con mayor privilegio (ADMIN y COORDINADOR) utilizan firmas digitales RSA-PSS y certificados X.509, los voluntarios y operativos acceden con un método simplificado de correo y contraseña. El diseño visual basado en semáforos de estado y la automatización de procesos complejos, como la sincronización del Administrador Espejo y la expiración por `end_date`, permiten que la organización mantenga un entorno seguro de forma autónoma.

3. Solución Propuesta y Arquitectura

El sistema implementa una arquitectura de seguridad por capas basada en los principios de defensa en profundidad y mínimo privilegio. Bajo este enfoque, la seguridad del sistema sigue una serie de filtros consecutivos y complementarios estructurados tanto a nivel de red como criptográfico y de aplicación. En este sentido, si un atacante logra vulnerar o evadir una de las capas exteriores, los controles de las capas subsiguientes detienen la progresión de la amenaza, protegiendo el núcleo del sistema donde reside la base de datos con la información sensible.

3.1. Estructura de las Capas de Seguridad

A continuación se detalla la función matemática, algorítmica y lógica de cada una de las capas que componen la arquitectura del Gestor de Identidades, desde el punto de entrada del cliente hasta la persistencia de los datos.

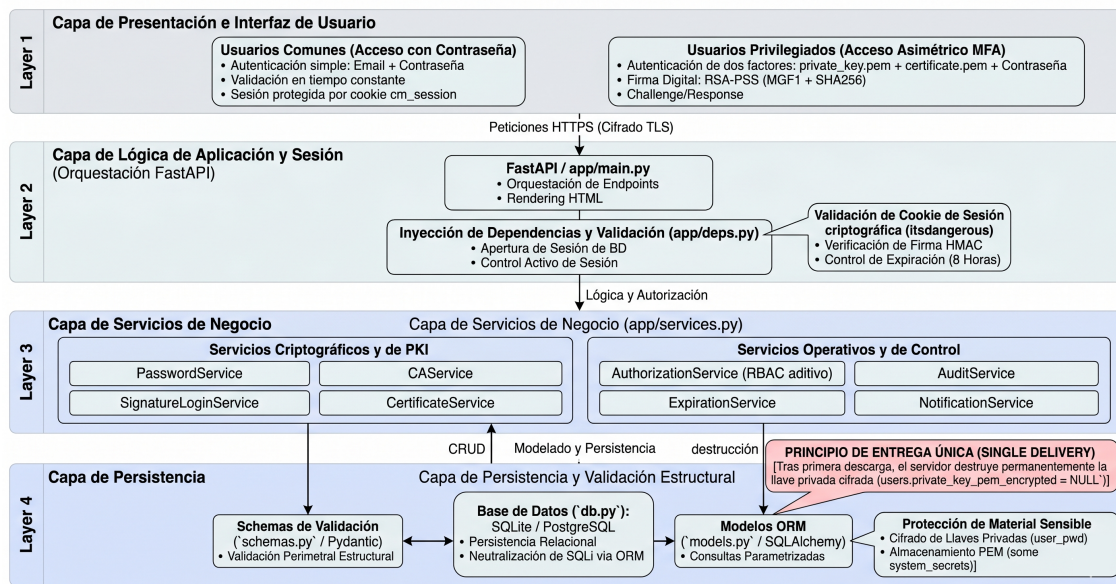


Figura 1: Arquitectura del sistema (elaboración propia)

3.1.1. Capa 1: Autenticación Bifurcada y Puerta de Entrada

Es el único punto de contacto con el exterior y actúa como el filtro perimetral de identidad aplicando un esquema estrictamente bifurcado según el nivel de riesgo del rol asignado. Los usuarios comunes (OPERATIVO, VOLUNTARIO) se autentican mediante un esquema simple basado en algo que el usuario sabe, requiriendo únicamente su correo electrónico y su contraseña. El acceso es validado comparando el hash PBKDF2 de la contraseña ingresada contra el guardado en la base de datos utilizando comparaciones en tiempo constante (`hmac.compare_digest`).

Por otro lado, para los usuarios privilegiados (ADMIN, COORDINADOR) se implementa un esquema obligatorio de dos factores basado en posesión criptográfica asimétrica. El usuario debe proporcionar su contraseña de descifrado y cargar dos archivos independientes en formato PEM: su archivo de llave privada (`private_key.pem`) y su certificado público (`certificate.pem`). El backend valida el número de serie, el correo incrustado en la extensión SAN (Subject Alternative Name), las ventanas de vigencia y la firma de la cadena de confianza. Finalmente, el servidor envía un desafío dinámico (*challenge*) compuesto por un nonce y una marca de tiempo, el cual debe ser firmado digitalmente mediante el algoritmo RSA-PSS (con esquema de máscara MGF1 y hashing SHA-256) utilizando la llave privada del usuario, validando matemáticamente la firma en el backend con la llave pública del certificado para autorizar la sesión.

3.1.2. Capa 2: Control de Acceso Basado en Roles (RBAC)

Una vez que el usuario demuestra su identidad de forma inequívoca en el perímetro del sistema (Capa 1), el servidor genera y gestiona una sesión web segura. La Capa 2 se encarga de limitar estrictamente qué acciones y recursos puede consumir ese usuario en función de sus competencias autorizadas, aplicando el principio de Mínimo Privilegio. Esta validación se divide en dos frentes complementarios: la autenticación de la sesión mediante cookies criptográficas y la autorización granular basada en roles (RBAC).

A. Mecanismo de Sesión Segura mediante *itsdangerous* A diferencia de las arquitecturas que transmiten tokens expuestos como JWT independientes, el sistema delega el resguardo del estado del usuario en cookies de sesión firmadas criptográficamente en el servidor para mitigar riesgos de interceptación de credenciales. Tras un inicio de sesión exitoso, el backend utiliza la clase `URLSafeTimedSerializer` provista por la biblioteca *itsdangerous* para instanciar un serializador acoplado a un secreto global de alta entropía (`session_secret`) y un salt específico denominado `cm-session`. Este componente genera un token compacto e inmutable estructurado bajo el formato estándar de tres secciones de texto seguro:

$$\text{Token} = \text{payload_base64_urlsafe} \cdot \text{timestamp_base64} \cdot \text{firma_hmac_base64}$$

El cuerpo del mensaje (*payload*) transporta únicamente la información mínima de identificación del usuario mediante su identificador primario (`{\"uid\": user_id}`). La firma criptográfica se calcula utilizando un algoritmo HMAC-SHA256, lo que impide que un atacante externo altere los privilegios del token sin invalidar matemáticamente la firma completa. Posteriormente, el servidor inyecta este elemento criptográfico en el navegador web del cliente bajo la cookie bautizada como `cm_session`, la cual cuenta con atributos restrictivos de seguridad activa:

- `httponly=True`: Restringe el acceso a la cookie exclusivamente al canal de transporte HTTP, impidiendo de forma absoluta que scripts maliciosos en el navegador (vectores de ataque XSS) puedan leer o exfiltrar el identificador de sesión.
- `secure=True`: Obliga al navegador a transmitir la cookie únicamente bajo conexiones cifradas mediante protocolos TLS/HTTPS en entornos productivos.
- `samesite=\"lax\"`: Mitiga los vectores de ataque de Falsificación de Petición en Sitios Cruzados (CSRF), controlando que la cookie de sesión no sea adjuntada de manera automática en solicitudes originadas desde dominios externos o de terceros.
- **Expiración de la sesión (`_SESSION_MAX_AGE`)**: El ciclo de vida de la cookie está estrictamente acotado a un horizonte temporal de 8 horas (28,800 segundos), reduciendo drásticamente la ventana de oportunidad para el secuestro de sesiones en terminales físicos compartidos.

B. Intercepción de Peticiones y Control de Acceso Basado en Roles (RBAC) Para asegurar que las políticas de acceso se ejecuten sin excepciones, el componente de inyección de dependencias nativo de FastAPI (`app/deps.py`) intercepta obligatoriamente la cookie de sesión en cada petición HTTP entrante. El backend lee el valor de `cm_session`, verifica la firma HMAC en tiempo constante y corrobora las marcas de tiempo frente al límite de expiración configurado. Simultáneamente, el interceptor realiza una validación lógica cruzada en la base de datos para constatar que la cuenta del actor se encuentre en un estado válido, denegando el flujo de manera inmediata si el estatus del usuario ha sido cambiado a `revoked` o `expired`.

Si la sesión es íntegra, toma el control el módulo `AuthorizationService`, el cual evalúa los permisos granulares (*scopes*) mediante un esquema RBAC de carácter estrictamente aditivo. La arquitectura del software rechaza el modelo de herencia de roles; en su lugar, la función `AuthorizationService.authorize(db, user, resource, action)` realiza un JOIN explícito en la base de datos relacional entre las entidades de usuarios, roles y la tabla asociativa `role_permissions`. De este modo, un usuario solo está facultado para ejecutar operaciones que se encuentren explícitamente mapeadas a su rol. La matriz jerárquica de permisos granulares está limitada de la siguiente manera:

- **ADMIN:** Perfil de máxima confianza. Faculta el control absoluto sobre el catálogo de usuarios (`users: create, view, activate, revoke, change_role, change_expiration`), la consulta de la bitácora forense de auditoría (`audit: view`) y la activación de los protocolos de recuperación criptográfica de emergencia.
- **COORDINADOR:** Rol operativo privilegiado. Permite la visualización y edición avanzada de documentación interna (`documents: view, edit`), el monitoreo de operaciones de campo (`operations: view`) y la consulta de su material criptográfico personal (`certificates: view`).
- **OPERATIVO:** Perfil operativo básico. Restringido a la visualización pasiva de documentos (`documents: view`) y a la inserción o edición de registros de campo en operaciones generales (`operations: view, edit`).
- **VOLUNTARIO:** Rol de mínimo impacto. Limitado exclusivamente a la consulta y lectura de documentos autorizados para tareas humanitarias (`documents: view`).

Si un colaborador intenta consumir un servicio para el cual su rol no cuenta con un permiso explícito (por ejemplo, si un usuario con perfil **OPERATIVO** dispara una solicitud HTTPS hacia el endpoint de revocación `/users/revoke`), la inyección de dependencias detiene el procesamiento en el perímetro del backend. El sistema frena la petición de forma atómica, devuelve un código de estado HTTP 403 `Forbidden` y genera un registro síncrono inmutable a través del `AuditService` detallando la identidad del actor, la dirección IP de origen, el agente de usuario y el recurso comprometido, garantizando así la trazabilidad forense y el principio de no repudio.

3.1.3. Capa 3: Validación Estructural de Datos y Contratos de Entrada

Esta capa impide el procesamiento de datos maliciosos o mal formateados que busquen corromper la memoria del servidor o inyectar código en la persistencia. Antes de tocar las funciones de negocio o criptografía, los parámetros de entrada se someten a los modelos de validación de `schemas.py`. Aquí se inspecciona rigurosamente la entropía de las contraseñas, formatos estrictos de correos electrónicos y tipos de datos. Si la entrada no cumple el contrato, el framework la rechaza en el perímetro. La interacción con la base de datos se realiza a través de objetos de Python en `models.py`. Al no utilizar concatenación de cadenas de texto plano para construir consultas SQL, el uso del ORM parametriza automáticamente todas las variables, neutralizando por completo el vector de ataque de Inyección SQL (SQLi).

3.1.4. Capa 4: Persistencia y Bóveda Criptográfica (Datos en Reposo)

Es la capa más profunda de la arquitectura, encargada de la persistencia de datos mediante el ORM `SQLAlchemy` mapeado hacia bases de datos relacionales `SQLite` (generando el archivo `identity_demo.db` en entornos locales) o `PostgreSQL` para producción. Para el alcance del entorno de demostración, los secretos globales de la organización (como la llave privada de la CA raíz y el certificado raíz) se almacenan como texto plano en formato PEM dentro de la tabla dedicada `system_secrets`. Sin embargo, el sistema protege activamente las llaves privadas de los usuarios privilegiados (**ADMIN** y **COORDINADOR**) guardándolas en un estado cifrado con la contraseña del propio usuario.

Una vez que el usuario realiza la descarga de su credencial, el servidor ejecuta una rutina atómica que destruye el registro binario del servidor asignando el campo `private_key_pem_encrypted` a nulo, aplicando la filosofía de entrega única. Adicionalmente, el entorno se configura con la clase `Settings` de Pydantic, la cual actúa como salvaguarda activa impidiendo el arranque del servidor en entornos de producción si se detecta que las claves secretas conservan los valores por defecto.

3.2. Flujo de una Petición a Través de las Capas (Caso de Éxito vs Error)

Para visualizar la efectividad de la arquitectura, se puede analizar matemáticamente el recorrido de una solicitud HTTPS para la acción crítica de Revocación de un Usuario:

1. **Recepción (Filtro Perimetral):** El API Gateway en `app/main.py` recibe la petición HTTPS con el identificador del objetivo.
2. **Evaluación de Capa 1 y 2 (Seguridad de Sesión):** `app/deps.py` extrae la cookie de sesión. Descifra y verifica la firma HMAC del token; si el token fue manipulado o ya expiró por tiempo, la petición muere aquí devolviendo un error 401 `Unauthorized`. Si es correcto, extrae el perfil del actor.
3. **Evaluación de Capa 2 (Autorización):** El `AuthorizationService` verifica en la base de datos si el rol del actor cuenta con el permiso explícito para revocar. Si un rol no autorizado (como un voluntario) ejecuta la petición, el sistema bloquea el flujo y responde 403 `Forbidden`. Si el actor es un “Administrador”, la petición avanza.
4. **Evaluación de Capa 3 (Validación de Datos):** Pydantic valida que el ID del usuario a revocar cumpla con el tipo de dato esperado.
5. **Ejecución en Capa 4 (Persistencia):** El servicio crítico actualiza el campo del usuario a `revoked` y confirma el cambio mediante un `commit` en SQLAlchemy, asegurando la integridad referencial de los datos.
6. **Cierre de Auditoría:** Una vez completada la transacción, el `AuditService` escribe un registro inmutable en la bitácora (`audit_logs`) documentando la IP de origen, el timestamp, el actor de la acción y el resultado exitoso, garantizando el principio de no repudio.

3.3. Justificación

El sistema fue concebido bajo la premisa de que una organización de derechos humanos como Casa Monarca, al manejar información altamente sensible de poblaciones vulnerables, requiere un nivel de protección equivalente al del sector bancario o gubernamental como el SAT, pero adaptado a una realidad operativa de bajo costo y alta usabilidad. Se detallan las decisiones técnicas que se realizaron, las cuales se justifican bajo los supuestos propuestos a continuación.

3.3.1. Viabilidad de Infraestructura: Despliegue en Alojamiento Compartido (HostGator) vs. Nube Compleja (AWS)

Una de las decisiones más críticas en el diseño de software es el entorno de despliegue. Aunque la tendencia industrial actual favorece el uso de proveedores de nube hiperescalables como AWS, Azure o GCP, para el contexto de Casa Monarca se optó por un despliegue en un entorno de hosting compartido tradicional (HostGator) utilizando scripts de automatización vía SSH. Esto se debe principalmente para reducir costos operativos, ya que las organizaciones no gubernamentales (ONG) operan bajo presupuestos estrictos y donaciones variables. Los esquemas de precios de AWS se basan en el consumo como ancho de banda, lecturas/escrituras de base de datos, tiempo de cómputo de instancias EC2 y almacenamiento, lo que introduce una alta volatilidad financiera y el riesgo de cobros imprevistos debido a picos de tráfico o ataques de denegación de servicio (DoS). HostGator, en contraste, ofrece una estructura de costos fijos mensuales o anuales sumamente bajos, permitiendo a la organización planificar financieramente a largo plazo sin desviar fondos de su misión humanitaria.

Otra consideración es la simplicidad en el mantenimiento, debido a que el uso de infraestructuras en la nube complejas exige la adopción de herramientas de orquestación, contenedores y automatización pesada como Docker, Kubernetes, Ansible o Terraform. Mantener esta cadena de herramientas (*toolchain*) requiere personal de ingeniería altamente especializado en DevOps de forma permanente. Al empaquetar el sistema como una aplicación Python/FastAPI nativa capaz de ejecutarse sobre un entorno básico con scripts SSH automatizados,

se elimina la dependencia de un departamento de TI avanzado. La organización puede mantener el sistema bajo un modelo de administración simplificado, garantizando la continuidad operativa a lo largo de los años sin caer en la obsolescencia por falta de personal técnico.

3.3.2. Robustecimiento del Esquema de Autenticación mediante Bifurcación de Acceso (RBAC)

La arquitectura no aplica las mismas medidas de seguridad a todos los usuarios, sino que implementa una separación estricta de mecanismos de autenticación según el nivel de privilegio y la superficie de riesgo del rol que se asignó. De esta manera, se mitiga el phishing y se evitan que las credenciales tengan algún tipo de vulnerabilidad en roles críticos. Los ataques de ingeniería social y phishing son responsables de la gran mayoría de las brechas de seguridad a nivel mundial. Si un atacante logra robar la contraseña de un usuario común (como un voluntario), el impacto se autolimita gracias al Control de Acceso Basado en Roles (RBAC), el cual restringe sus permisos exclusivamente a visualizaciones de bajo impacto. Por esta razón, para los roles OPERATIVO y VOLUNTARIO, un esquema de correo y contraseña robustecida con PBKDF2-HMAC-SHA256 e inyección de cookies seguras (HttpOnly, SameSite=Lax) es técnicamente proporcional y suficiente.

Sin embargo, para los roles con capacidad de alterar la configuración del sistema o exfiltrar bases de datos completas (ADMIN y COORDINADOR), las contraseñas tradicionales son un mecanismo de defensa inaceptable. Al obligar a estos roles a autenticarse mediante un protocolo de desafío-respuesta con llaves asimétricas RSA de 2048 bits y certificados X.509, la seguridad pasa a depender estrictamente de algo que el usuario posee. En este sentido, un atacante externo, incluso si intercepta la contraseña de descifrado local del coordinador, no podrá iniciar sesión si no posee el archivo digital `.pem`, reduciendo a cero el riesgo de intrusiones por ataques de diccionario o phishing remoto.

3.3.3. Adopción de Estándares Algorítmicos Criptográficos e Industriales

La selección del ecosistema tecnológico y criptográfico responde a estándares internacionales avalados por agencias de ciberseguridad como el NIST y el cumplimiento normativo del marco legal mexicano. Por ejemplo, la elección de FastAPI como framework principal responde tanto a su alto rendimiento en milisegundos como a su integración nativa con la validación de datos a través de Pydantic. Esto garantiza que cualquier intento de inyección de desbordamiento de búfer o envío de parámetros corruptos sea filtrado en el perímetro de la API. Por su parte, SQLAlchemy funciona como una capa de abstracción sobre MySQL. Al mapear las entidades de seguridad como objetos de Python y parametrizar de forma obligatoria todas las consultas a las tablas (`users`, `audit_logs`, etc.), se elimina por completo la posibilidad de ataques de Inyección SQL (SQLi), una de las técnicas más comunes utilizadas por atacantes para secuestrar bases de datos.

Se prefirió el uso de RSA de 2048 bits por su altísima compatibilidad con almacenes de certificados estándar. Para el inicio de sesión se implementó obligatoriamente el acolchado PSS (Probabilistic Signature Scheme) en lugar del clásico PKCS#1 v1.5. PSS introduce una sal aleatoria en cada operación de firma, lo que significa que el mismo desafío matemático firmado dos veces generará firmas completamente distintas, protegiendo al sistema contra ataques de criptoanálisis por texto elegido y ataques de repetición (*replay attacks*). Por otro lado, el uso de AES-256 (Advanced Encryption Standard con llaves de 256 bits) provisto por cryptography asegura que los datos en reposo dentro de MySQL cuenten con cifrado de grado militar. Si un tercero sustrae físicamente los discos duros del servidor o exfiltra los archivos de respaldo del hosting, se topará con bloques de datos indistinguibles del ruido aleatorio, blindando la confidencialidad de la población migrante atendida.

3.3.4. Filosofía del Ciclo de Vida: El Principio de Entrega Única (Single Delivery)

El almacenamiento permanente de llaves privadas en un servidor centralizado es propenso a amenazas, ya que si el servidor sufre una intrusión y el atacante obtiene privilegios de superusuario (*root*), podría descifrar las llaves de todos los coordinadores de la organización. Para neutralizar este riesgo estructural, el sistema implementa la política de Entrega Única. El servidor funciona únicamente como una fábrica temporal de material criptográfico. En el instante en que el administrador genera un usuario y descarga su llave privada empaquetada, el backend ejecuta una rutina que destruye el registro binario en la base de datos, dejando el campo `private_key_pem_encrypted` estrictamente en nulo. A partir de ese segundo, la llave privada solo existe en el dispositivo físico del coordinador.

Esta decisión de diseño traslada la responsabilidad de la custodia al eslabón final y garantiza de forma matemática el principio de No Repudio: si se registra una acción firmada digitalmente en la bitácora de auditoría, la

organización tiene la certeza absoluta de que solo pudo haber sido ejecutada por el poseedor de esa llave física, imposibilitando que incluso los propios administradores del sistema puedan falsificar la firma de un coordinador.

3.4. Implementación Técnica y Estructura del Software

3.4.1. BackEnd

El backend, orquestado mediante FastAPI, actúa como un API Gateway interno que intercepta todas las peticiones HTTPS, gestionando la serialización de datos y evitando que errores técnicos filtren información sensible. Los componentes y servicios críticos (`app/services.py`) se dividen en las siguientes responsabilidades:

Servicio de Inicio de Sesión Criptográfico (SignatureLoginService) Es el componente backend con mayor rigurosidad criptográfica. Es el encargado de procesar la autenticación de dos factores para los roles privilegiados (ADMIN y COORDINADOR).

- **Validación ASN.1 del Certificado:** Parsea en memoria el archivo `certificate.pem` cargado por el usuario, comprobando el número de serie único almacenado en la base de datos y corroborando que el tiempo del sistema esté dentro de las ventanas `not_valid_before_utc` y `not_valid_after_utc`.
- **Verificación de Identidad Cruzada:** Extrae el campo Subject Alternative Name (SAN) con formato de correo electrónico (RFC822Name) y valida que coincida exactamente con la identidad del usuario que solicita el acceso.
- **Procesamiento del Desafío (Challenge-Response):** Genera un nonce dinámico (un bloque de datos único que incluye el ID del usuario y un timestamp). El servicio utiliza la llave pública extraída del certificado para verificar la firma digital enviada por el cliente (calculada con el algoritmo asimétrico RSA-PSS con máscara MGF1 y SHA-256). Si la verificación matemática pasa, se demuestra la posesión física de la llave privada y se autoriza la sesión.

Servicio de Autoridad Certificadora Interna (CertificateAuthorityService) Es el núcleo de confianza (*Root of Trust*) de la organización.

- **Gestión de la CA Raíz:** Se encarga de inicializar de manera persistente en el almacén de secretos del sistema (`system_secrets`) el par de llaves asimétricas y el certificado X.509 autofirmado de la CA con una vigencia extendida de 10 años.
- **Emisión de Credenciales de Coordinadores:** Utiliza la llave privada de la CA para firmar digitalmente los certificados emitidos a los coordinadores, estableciendo la propiedad `issuer_name` con los datos de la CA del sistema, inyectando las extensiones obligatorias de restricción `BasicConstraints(ca=False)` y los usos permitidos (`KeyUsage.digital_signature` y `KeyUsage.key_encipherment`).

Servicio de Auditoría e Inmutabilidad (AuditService) Este es el encargado del cumplimiento del principio de No Repudio dentro del sistema.

- **Registro de Eventos Críticos:** Captura de forma síncrona metadatos detallados de cada petición exitosa o fallida (como intentos de login por contraseña, logins criptográficos, revocaciones de usuarios, accesos denegados o expiraciones automáticas).
- **Persistencia Segura:** Escribe los eventos en la tabla relacional `audit_logs`, registrando de manera estructurada campos como: `actor_user_id`, `target_user_id`, `action`, `resource`, `result`, `ip_address` y `user_agent`. Este flujo cierra el ciclo de seguridad, permitiendo una trazabilidad forense completa si se detectan anomalías.

Servicio de Expiración Automática (ExpirationService) Es el proceso asíncrono que actúa de manera proactiva para reducir la carga administrativa y mejorar la seguridad de la plataforma.

- **Evaluación de Vigencia Lógica:** Examina periódicamente el campo `end_date` de los usuarios cuyo estatus actual sea `active`.

- **Inactivación Atómica:** Si la marca de tiempo del servidor supera la fecha de finalización configurada, el servicio cambia automáticamente el estatus del usuario a *expired*, bloqueando inmediatamente cualquier intento de uso de sus tokens o certificados activos.

3.4.2. FrontEnd

La interfaz web está diseñada para permitir al personal humanitario de Casa Monarca interactuar ágilmente con los servicios criptográficos subyacentes sin enfrentarse a barreras o complejidades tecnológicas complejas. Los módulos más relevantes en la presentación son:

Panel de Control de Identidades (Dashboard de Administración) Es la consola centralizada de supervisión reservada para el rol de ADMIN.

- **Control Visual de Estados:** Implementa códigos de color dinámicos basados en semáforos para los estados de los colaboradores (*active* verde, *pending* amarillo, *revoked* rojo, *expired* gris), facilitando auditorías visuales e inspecciones rápidas del estatus del personal por parte del administrador.
- **Ocultamiento de Acciones Críticas por RBAC:** La interfaz web renderiza los componentes interactivos de manera condicional. Los botones para funciones altamente sensibles, tales como “Revocar Usuario”, “Cambiar Rol” o “Crear Usuario”, solo se inyectan en el DOM si el token de sesión descifrado confirma el rol jerárquico de Administrador. Esto limita la superficie de ataque y previene el error humano por parte de operadores con menores privilegios.

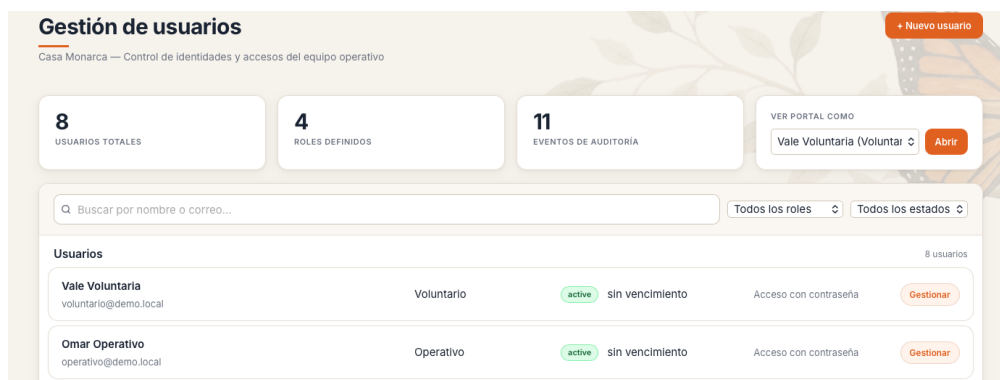


Figura 2: Panel de Control del administrador (elaboración propia)

Módulo de Autenticación Bifurcado (Formularios de Inicio de Sesión) Presenta una interfaz de acceso adaptada al nivel de riesgo del usuario.

- **Formulario Estándar:** Para voluntarios y operativos, ofrece campos limpios de entrada de texto para correo electrónico y contraseña.
- **Formulario de Carga Criptográfica:** Para roles privilegiados, se transforma en una interfaz de carga de archivos de arrastrar y soltar (*drag-and-drop*). Solicita explícitamente tres componentes esenciales: el archivo de certificado (*certificate.pem*), el archivo de llave privada (*private_key.pem*) y la contraseña de paso utilizada para descifrar localmente la llave empaquetada.

Portal de Autoservicio y Descarga de Certificados (User Portal) Módulo seguro al que tienen acceso los coordinadores validados.

- **Descarga Única de Llave Privada:** Aloja el componente interactivo que permite al usuario descargar por única vez la llave privada cifrada generada por el servidor. La interfaz despliega advertencias críticas informando que el archivo se borrará permanentemente de los servidores tras confirmar la descarga, reforzando la cultura de custodia digital.

Archivo de acceso

Solo para Administrador y Coordinador

**Arrastra tu archivo aquí**
o selecciona un archivo

Certificado de identidad

Solo para Administrador y Coordinador

**Arrastra tu certificado aquí**
o selecciona un archivo

Figura 3: Solicitud de credenciales (elaboración propia)

- **Consulta de Certificado Propio:** Permite al colaborador visualizar las extensiones de su certificado X.509 activo, verificar su fecha límite de vigencia y descargar el certificado público tantas veces como sea necesario para la firma de documentos institucionales.

Mi cuenta

Bienvenido/a, Omar Operativo

Información personal

NOMBRE COMPLETO

Omar Operativo

CORREO ELECTRÓNICO

operativo@demo.local

ROL

Operativo

ESTADO

Activo

OPERATIVO

Puedes crear y consultar registros. Revisa los datos ingresados y canaliza al nivel coordinador para actualizaciones o acciones adicionales.

CR — Crear y Leer

Figura 4: Portal del Usuario (elaboración propia)

3.5. Conexión de los Componentes

La arquitectura sigue un flujo de comunicación directo y seguro empezando por la orquestación donde FastAPI recibe las peticiones desde el FrontEnd mediante formularios HTML y parámetros de consulta. Posteriormente se requerirá la validación de identidad, donde antes de procesar cualquier lógica, el sistema utiliza dependencias (`deps.py`) para inyectar la sesión de la base de datos y validar quién realiza la acción. Los datos se almacenan en un esquema relacional definido en SQL que vincula usuarios con roles y permisos a través de llaves foráneas, garantizando la integridad referencial de toda la estructura de identidades. Cada interacción exitosa o denegada en el FrontEnd dispara una llamada al `AuditService` en el BackEnd, cerrando el ciclo de seguridad y control.

4. Resultados

La implementación del Gestor de Identidades dio como resultado una plataforma funcional que integra seguridad criptográfica con una gestión administrativa ágil. Los resultados se dividen en los logros alcanzados en el servidor, la interfaz y la validación operativa.

4.1. Estructura de la solución

RetoCriptografia-Equipo2/

```

app/
  __init__.py
  config.py
  db.py
  deps.py
  main.py
  main2.py
  models.py
  schemas.py
  services.py
docs/
  USO_Y_FUNCIONAMIENTO.md
sql/
  schema.sql
.env.example
.gitignore
pyproject.toml
README.md

```

4.2. Resultados del BackEnd y API

Se logró consolidar una infraestructura robusta basada en FastAPI, capaz de procesar peticiones de identidad en milisegundos. Se consolidó el servicio `SignatureLoginService` para usuarios privilegiados (ADMIN y COORDINADOR). Este procesa y valida de forma exitosa firmas digitales generadas con el esquema probabilístico avanzado RSA-PSS (utilizando máscara MGF1 y SHA-256) sobre un *challenge* dinámico generado por el servidor. La verificación de los hashes de contraseñas (derivadas mediante PBKDF2-HMAC-SHA256 con 120,000 iteraciones) se ejecuta mediante `hmac.compare_digest`. Esto garantiza comparaciones en tiempo constante, anulando la posibilidad de que un atacante deduzca credenciales analizando microsegundos de respuesta.

El `AuditService` registra de manera síncrona metadatos estructurados (IP, timestamp, actor, acción y recurso) en la tabla `audit_logs` ante cualquier interacción crítica, blindando la trazabilidad del sistema, lo cual cumple con el principio de no repudio. Además, el backend se integró exitosamente con MySQL/SQLite donde la información sensible y el material PKI temporal se almacenan bajo el estándar de cifrado simétrico AES-256-CBC, cumpliendo con el marco legal mexicano de protección de datos.

4.3. Resultados del FrontEnd e Interfaz

La interfaz web construida permite al personal operativo y humanitario de Casa Monarca interactuar de forma intuitiva con el backend criptográfico sin requerir conocimientos técnicos avanzados. Se pudo establecer un control visual de los estados, donde se implementó un código de colores dinámico basado en semáforos para la supervisión rápida del estatus de los colaboradores (active: verde, pending: amarillo, revoked: rojo, expired: gris).

Además, gracias a la inyección de dependencias y la lectura del token de sesión, los componentes interactivos críticos (botones como “Revocar”, “Cambiar Rol” o “Crear Usuario”) se ocultan del DOM si el usuario no posee el rol de Administrador, limitando drásticamente la superficie de error humano. Por otro lado, se obtuvo una interfaz de inicio de sesión bifurcada que despliega una entrada simple de credenciales para usuarios comunes, y un módulo de carga de archivos de arrastrar y soltar (*drag-and-drop*) que solicita el certificado (`.pem`), la llave privada (`.pem`) y su clave de descifrado para accesos privilegiados. Por último, el portal de autoservicio de descarga única permitió un espacio seguro donde los coordinadores pueden descargar su certificado público y realizar la descarga única de su llave privada empaquetada bajo el estándar PKCS#8.

4.4. Resultados de la Solución Completa

Al integrar de extremo a extremo el BackEnd criptográfico con la interfaz del FrontEnd, se garantiza, en primer lugar, el principio de entrega única: cuando el Administrador genera un usuario privilegiado y este descarga su llave privada por primera vez, el sistema ejecuta una rutina atómica que destruye el registro binario en el servidor, asignando el campo `private_key_pem_encrypted` a nulo. A partir de ese instante, la llave privada solo existe

en el dispositivo físico del coordinador, blindando al sistema contra exfiltraciones masivas si la base de datos central es compromised.

Por otro lado, el `ExpirationService` evalúa de forma asíncrona el campo `end_date` de los usuarios activos. Si la vigencia de la cuenta ha caducado, cambia su estado a `expired`. El backend rechaza automáticamente cualquier intento de autenticación si las vigencias del certificado X.509 (`not_after_utc`) y la fecha lógica de la base de datos no se encuentran perfectamente sincronizadas, resolviendo eficientemente la gestión de revocaciones. La clase `Settings` (basada en `Pydantic`) actúa como una salvaguarda activa al validar la configuración del sistema. Si el software se despliega con el entorno marcado como `production`, el validador estructural genera un fallo crítico e impide que el servidor backend arranque si la clave `SESSION_SECRET` conserva su valor por defecto de desarrollo, forzando la seguridad de la infraestructura desde el diseño.

4.5. Validación mediante Casos de Prueba

Para asegurar la fiabilidad de la solución, se sometió al sistema a una batería exhaustiva de pruebas funcionales y de seguridad. Se validaron procesos críticos como el Alta de usuario con datos válidos, la denegación de accesos por contraseñas incorrectas o por usuarios no activos (estados `pending` o `revoked`), y la correcta visualización de la bitácora de auditoría por parte del administrador. Estas pruebas confirmaron que el sistema rechaza correctamente operaciones duplicadas o incompletas y que responde con los códigos de error adecuados (401 y 403) ante intentos de acceso no autorizados, garantizando la integridad del flujo de identidades.

A continuación se presenta un resumen de los casos de prueba ejecutados exitosamente:

Tabla 1: Resumen de validaciones (elaboración propia)

Caso de Uso	Descripción	Resultado Esperado	Estatus
Alta Válida	Creación de usuario con datos únicos.	Usuario creado y certificado emitido.	Exitoso
Login Incorrecto	Intento de acceso con credenciales erróneas.	Rechazo de login y registro en auditoría.	Exitoso
Revocación	Cambio de estado de un usuario activo a revocado.	Acceso denegado inmediatamente tras el cambio.	Exitoso
Expiración	Cambio automático al llegar a <code>end_date</code> .	Usuario marcado como <code>expired</code> sin intervención.	Exitoso
Auditoría	Consulta de bitácora como administrador.	Despliegue de eventos recientes de seguridad.	Exitoso

5. Conclusiones

La culminación de la primera etapa del Gestor de Identidades para Casa Monarca demuestra que es técnicamente viable y operativamente eficiente diseñar un sistema que equilibre la alta seguridad criptográfica con la usabilidad humanitaria. Al alinear el desarrollo con el marco normativo mexicano (Derechos ARCO y la Ley Federal de Protección de Datos Personales), se blindó la confidencialidad de la información sensible perteneciente a la población migrante atendida sin imponer barreras tecnológicas complejas al personal de la organización. La arquitectura modular implementada en `FastAPI` y su despliegue simplificado en `HostGator` resolvieron de forma paralela los retos de integridad, autenticidad y no repudio, asegurando una solución de bajo costo y fácil mantenimiento a largo plazo.

Se cumplió el objetivo principal de erradicar riesgos como la suplantación de identidad o la alteración de datos mediante una Arquitectura de Seguridad por Capas. La bifurcación de roles (RBAC) permitió que las funciones críticas queden restringidas criptográficamente a través de llaves RSA-2048 y relleno probabilístico PSS, mientras que los operadores en campo interactúan con una interfaz web limpia y adaptada a sus capacidades técnico-operativas.

A través de la combinación atómica del `AuditService` y el Principio de Entrega Única (*Single Delivery*), el sistema se desligó de la custodia permanente de las llaves privadas de los coordinadores. Al destruirse la llave cifrada en el servidor inmediatamente después de su primera descarga, se garantiza de manera matemática el no repudio: cualquier acción registrada en las bitácoras inmutables (`audit_logs`) solo pudo haber sido firmada

por el poseedor físico de dicha llave, protegiendo al sistema incluso ante un compromiso total de la base de datos centralizada.

La decisión de rechazar infraestructuras de nube complejas (AWS/Docker) en favor de un entorno de hosting compartido e inyecciones de dependencias nativas demostró ser acertada. Se redujeron a cero los costos variables por consumo de ancho de banda y se eliminó la necesidad de contar con un equipo permanente de ingenieros DevOps, asegurando que la ONG pueda mantener la plataforma de forma autónoma. Las pruebas funcionales y de caja negra ejecutadas sobre el backend confirmaron que los filtros perimetrales interceptan y rechazan correctamente cualquier petición con firmas HMAC alteradas, certificados fuera de vigencia, o credenciales que no correspondan a la identidad del usuario (mediante la inspección estricta de la extensión SAN RFC822Name), respondiendo con los códigos estándares de la industria (401 y 403).

5.1. Trabajo Futuro

Para las siguientes etapas de desarrollo y escalabilidad del Gestor de Identidades, se plantea explorar la integración de tecnologías de Libro Mayor Distribuido (*Distributed Ledger Technology* o Blockchain) para el almacenamiento de los registros generados por el AuditService. Al migrar la tabla `audit_logs` desde un esquema relacional centralizado (MySQL) hacia una red de nodos permissionada, se elevaría la inmutabilidad de la auditoría forense a un nivel superior, impidiendo que incluso un atacante con privilegios de superusuario en la base de datos pueda alterar el historial de accesos o revocaciones.

Por otro lado es recomendable expandir los mecanismos de la Capa 1 de autenticación para integrar estándares modernos de autenticación web sin contraseña, tales como WebAuthn y FIDO2. Esto permitiría al personal humanitario en campo autenticar sus sesiones privilegiadas o firmar desafíos criptográficos utilizando factores biométricos nativos de sus dispositivos móviles (huella dactilar o reconocimiento facial), incrementando la seguridad asimétrica y simplificando drásticamente la experiencia de usuario al eliminar la manipulación manual de archivos `.pem` o contraseñas complejas.

También es importante diseñar e incorporar un submódulo dedicado a la emisión periódica de Listas de Revocación de Certificados (CRL) o un respondedor bajo el Protocolo de Estado de Certificados en Línea (OCSP). Esto dotará al sistema de un mecanismo estandarizado industrialmente para revocar la validez de los certificados X.509 de forma inmediata en tiempo real, sustituyendo la actual validación lógica cruzada basada en el campo `end_date` de la base de datos.

Por último, se ve viable evolucionar el esquema de cifrado de campos específicos mediante AES-256 hacia un modelo de Cifrado Homomórfico Parcial o esquemas de Base de Datos Cifrada de Extremo a Extremo (como *Always Encrypted*). Esto posibilitará que el motor de la base de datos realice búsquedas, ordenamientos y filtros analíticos sobre la información de los migrantes sin necesidad de descifrar los datos en la memoria RAM del servidor, neutralizando vectores de ataque basados en el volcado de memoria (*memory dumping*).

Referencias

- [1] Oracle. *Identity and Access Management User Lifecycle*.
- [2] Gobierno de México. *Ley de Firma Electrónica Avanzada*.
- [3] IBM. *Data Lifecycle Management and File Archiving*.

6. Anexos

6.1. Tabla de casos de prueba

Tabla 2: Evidencia de casos de pruebas del sistema (elaboración propia)

Caso	Descripción	Resultado	Evidencia
Alta de usuario válida	Admin crea usuario con nombre, correo único, rol, fecha opcional y contraseña .p12.	Usuario creado correctamente, certificado emitido, redirección al panel.	Alta de usuario válida
Alta con correo duplicado	Intentar crear usuario con correo ya existente.	Rechazo de la operación con error de validación.	Alta con correo duplicado
Alta con nombre vacío	Crear usuario con nombre en blanco o espacios.	Rechazo con error de validación.	Alta con nombre vacío
Alta con contraseña .p12 vacía	Crear usuario sin contraseña para .p12.	Rechazo con error de validación.	Alta con contraseña .p12 vacía
Alta por usuario sin permiso	Usuario no admin intenta crear usuario.	Acceso no autorizado.	Alta por usuario sin permiso
Activar usuario pending	Admin cambia estado de pending a active.	Estado actualizado a active.	Activar usuario pending
Revocar usuario activo	Admin cambia estado de active a revoked.	Estado actualizado a revoked.	Revocar usuario activo
Reactivar usuario revocado	Admin activa usuario que estaba revoked.	Estado pasa a active con acción de reactivación.	Reactivar usuario revocado
Login demo admin/admin	Ingresar con credenciales de bypass demo.	Login exitoso al dashboard admin.	Login demo admin/admin
Login .p12 exitoso	Correo válido + p12 correcto + contraseña correcta.	Login exitoso con identidad verificada.	Login .p12 exitoso
Login sin archivo .p12	Intentar login normal sin adjuntar archivo.	Rechazo con mensaje de archivo requerido.	Login sin archivo .p12
Cambiar expiración futura	Admin establece end date futura.	Fecha guardada correctamente.	Cambiar expiración con fec.
Cambiar expiración pasada	Admin intenta guardar expiración en fecha pasada.	Rechazo con error de validación.	Cambiar expiración con fec...
Usuario expira auto.	Usuario activo con fecha vencida al listar usuarios.	Estado cambia automáticamente a expired.	Usuario expira automática...
Reactivar por vigencia	Usuario expired recibe nueva fecha futura.	Estado vuelve a active.	Reactivar por vigencia
Cambio de rol válido	Admin cambia rol de un usuario.	Rol actualizado correctamente.	Cambio de rol válido
Login contraseña incorrecta	Subir .p12 válido con contraseña incorrecta.	Login rechazado.	Login con contraseña incorrecta
Login usuario no activo	Usuario en pending/revoked/expired intenta login.	Login rechazado por estado.	Login con usuario no activo
Certificado ajeno	Correo A + .p12 de usuario B.	Login rechazado por no correspondencia.	Login con certificado de otro usuario
Ver certificado propio	Usuario consulta vista/descarga de su certificado.	Acceso permitido.	Ver certificado propio
Auditoría admin	Admin consulta bitácora.	Respuesta correcta con eventos recientes.	Consultar auditoría como administrador
Usuario inexistente	Usuario que no existe intenta hacer login.	Usuario no registrado.	Login usuario inexistente

Tabla 2: Evidencia de casos de pruebas del sistema (elaboración propia)

Caso	Descripción	Resultado	Evidencia
Admin revocado	Admin revocado intenta login tras activación de admin espejo.	Contraseña incorrecta.	Archivo Admin Revocado